System Analysis and Design

Solving Problems and User Story

Why is software built?

- Software is often built to help the business clients or users solve their problems (called "pain points")
  - What is not working properly?
  - What is hurting them?
  - How serious is the pain?
- Example
  - Unable to respond to changing market conditions quickly
- The clients/users can often express their pain points, but not the solution

- Humans have pain points like - “I am bored”, “I need inspiration”
  - Many systems are built for these ***pain points***

What is the Problem?

- Software is built to address "pain/pleasure points"
- The better the **problem** can be stated/framed, the more likely it can be solved
- To ensure success, we need to pay proper attention to what is not working properly, what is hurting us
  - pain/pleasure points
- Once we identify the key high-level pain/pleasure points, further analysis will result in a detailed view of the problem
- Problem exists in a context
  - We need to understand the context to understand the "Pain Points"

How much do you understand the problem?

- To better understand the problem context, we should be familiar with the
  - Domain vocabulary (terminologies meaningful to the problem)
  - Activities, processes and tasks that require software solutions
- Examples of domain vocabulary:
  - “Credit”, “Debit”, “Account” and “Statement” all have a particular meaning in the banking domain
- Example of the process:
  - Enrolment

Actors

- What/who are people/things interacting with the software that we plan to develop?
  - In Software Engineering we call these people/things "**Actors**"
- Actors can have an impact on how the new software should operate
- Important to analyze these impacts when designing software solutions
  - Pain points of the clients/users must be addressed

Actors

- Actors play a role in the software system
  - A single ‘real human’ can take on multiple roles
  - An external system is considered an actor (iPod is an actor in the iTunes system)
  - If it interfaces with the system, then it can be considered an actor
- Every actor has different pain points
  - This means, the iPod (a non sentient entity) has pain points - - in its case it will be the interface to the iTunes software
  - iPod itself does not have the issue, but the developers of the iPod software will want a simple interface to iTunes

Actor Goals

- Every actor has a set of pain/pleasure points -- we build software to address these points
- Actors also have a 'GOAL' state in mind
  - Actors use systems to reach their 'GOAL' state
- Example GOALS:
  - Get the list of latest books on OOAD
  - Purchase a book from the online book store
  - View the latest video lectures on OOAD
- Software must allow the actor to achieve their 'GOALS'

What constraints are you dealing with?

- Constraints restrict your choices on how problems can be solved
- Example:
  - You must solve the problem in 2 months with $X
  - You are asked to improve performance of the current system, which was written in the Cobol language
  - The developed system must work in the client's operating environment, which is Linux
- Some other constraints could be
  - Laws/regulations
  - User's current training/education level

Problem domain

- Identify actors
- Understand the need/impact of each actor
- Write down the goals of each actor
- What are the constraints that we have to work with?
- What is the domain vocabulary?

Analysing the problem to figure out a direction of solving this problem ("**solution direction**")

Assumptions

- When we develop solutions, we often make assumptions on the problem domain
- *Example:* The users understand English, so the instructions given to the users are written in English
- Should we verify the validity of our assumptions?
  - Better do so, otherwise, we may waste our time in developing useless solutions for the clients/users

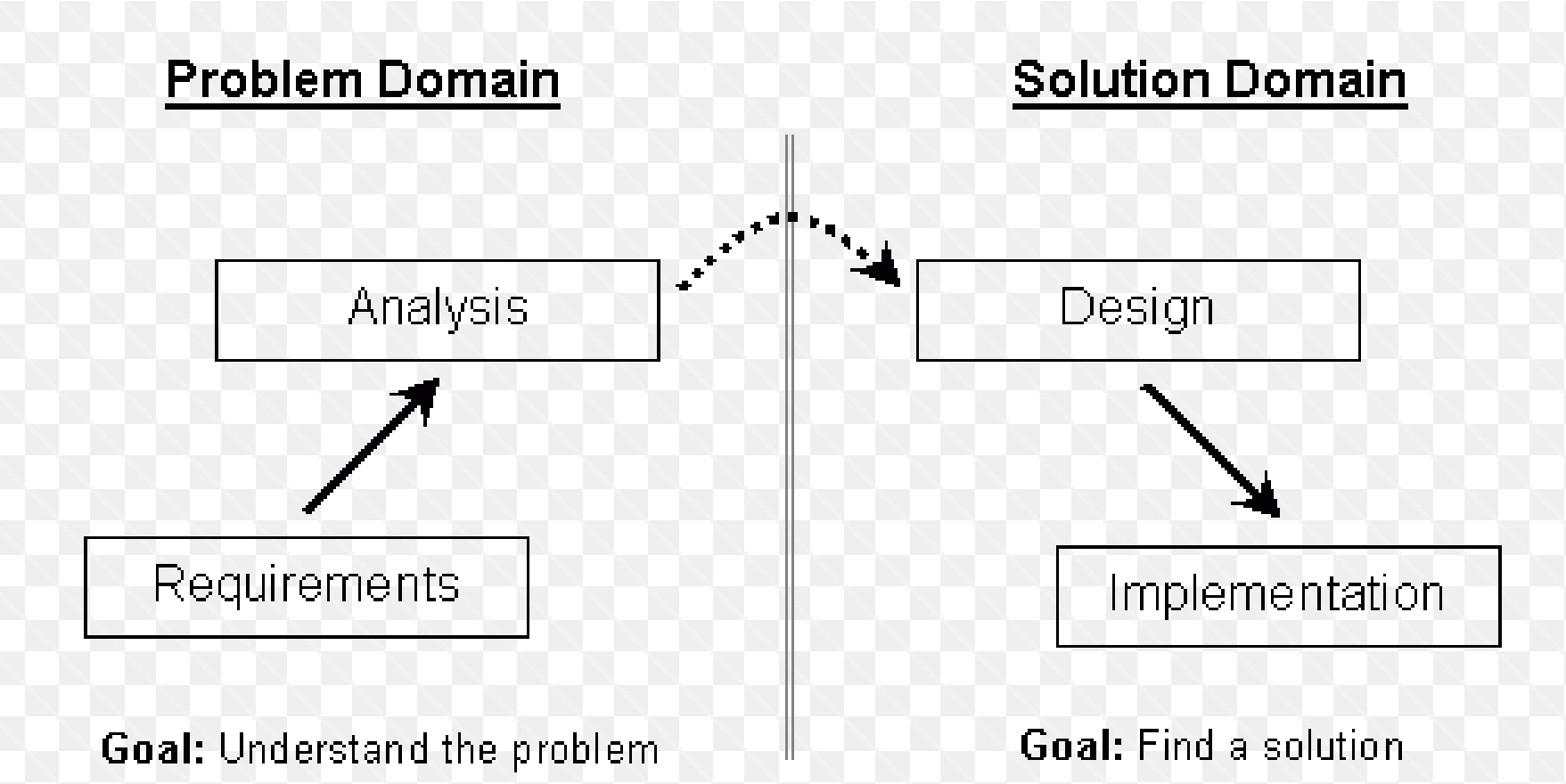
Solution development

- Choose the best solution out of potential solutions (requires decision making)
- Decision making is influenced by
  - Constraints (including some environmental factors)
  - Habits and Personality
- Questions to ask
  - Are these reasonable choices?
    - Can we defend the decisions/choices that we have made?
  - What is the impact of each choice?
    - Constraints of our next steps

Solution domain

- Initially, we may just have a solution direction
- Throughout the software development process, we keep refining the solutions based on
  - Our understanding of the problem domain
  - Previous decisions made for the solution
    - Note: Some of the decisions we made in the past will restrict what we do next

Problem domain vs Solution domain



Some useful statements

- In an early stage of software development, it is useful to have/make the following two statements
  - **Problem statement**
  - **Vision statement**

Problem statement

- Describes **what** needs to be done without describing **how**
- Agreed by all members in the project
- A description of a problem that helps the non-technical and technical personnel communicate
  - Technical terms should be explained if used

Problem statement

The problem of	people not being able to communicate properly as they get older

Vision statement

- It aims to
  - Cover the essence of the new system
  - Market the software
  - Keep the developers focused on the "**core**" essence of the system
- It should be brief
- <u>Example</u> -> **JU vision**:

At JU, we are passionate about excellence in higher education, vocational training and research. As an institution we are flexible in our teaching and learning, focused in our research and engaged with both industry and the community. We are committed to a sustainable JU.

Vision statement template

- The various parts of a general vision statement are:
  - For (target user, audience)
  - Who (statement of the need or opportunity)
  - The (product name) is a (product category)
  - That (key benefit, compelling reason to build/buy)
  - Unlike (primary competitive alternative)
  - Our product (statement of primary differentiation)

For	elderly or disabled persons with speech, hearing, and memory disabilities
Who	have difficulty communicating with others
The HOPE Cellular Phone Application (HCPA)	is a software application
That	provides the ability to communicate more effectively with others, and swiftly contact emergency responders through a simple, intuitive interface
Unlike	currently available systems that have poor interface to communicate with others and do not provide a detailed system for emergency services.
Our product	provides users with means of expressing themselves, and more easily understanding others. This is accomplished by text-to-speech and speech-to-text functionality and other features. The product also supports efficient means of reaching emergency contacts and emergency responders.

Vision statement template (cont.)

- Example one: Vision statement for the “CSE_Dynamics” game
  - **For**: CSE, JUstudents
  - **Who**: Are bored in between the lectures
  - **The**: CSE_Dynamics is a real-time strategy game
  - **That**: allows hours of interactive game play
  - **Unlike**: Warcraft
  - **Our**: CSE_Dynamics is custom designed to ensure that you can play with other CSE, JU students
- Once the key information is stated as above, it needs to be written up into a single paragraph like:
  - “CSE_Dynamics ***is a*** *{THE} real-time strategy game* ***for*** *(CSE, JU) students* ***who*** *are bored in between classes.* ***Unlike*** *Warcraft,* CSE_Dynamics ***is*** *{OUR} custom designed to {THAT} ensure hours of interactive game play between CSE, JUstudents.*”

Vision statement template (cont.)

- Example Two: Vision statement for the MJB online job-board:
  - **For**: Candidates
  - **Who**: Are seeking jobs
  - **The**: MJB is an online job board
  - **That**: Allows recruiters to post vacancies
  - **Unlike**: The classified section of the newspaper
  - **Our**: Job board allows for fast search and real-time access to positions
- The final vision statement for the MJB online job-board becomes
  - "The MJB **is** {THE} an online job board **for** candidates **who** are seeking jobs. **Unlike** the classified section of the newspaper, MJB **allows** {THAT} recruiters to post vacancies and allows {OUR} job hunters to fast search and real-time access to positions."

Gaps

- All projects work with gaps -- these gaps cause risks of project failures
- Three possible gaps
  - **K**nowledge gap - Do we know **what** to do?
  - **S**kill gap - Do we have the **skills** to do this?
  - **T**echnology gap – Do existing **technologies** help us solve our problem? Can we afford it?

Knowledge gap

- This gap exists when your knowledge of an issue or aspect is not complete
- Cause for inability to identify knowledge gaps
  - Over self-confident
  - The ego tends to hide some issues

Skill gap

- You know it, you have seen it done by others
- However, your skill for doing this is not good -- “insufficient practice”
- Example: “I do not know how long it will take to do X”
- Skills are difficult to develop without a certain level of practice
  - Again, ego tends to amplify your ability
- Should/Can all tasks be performed by experts?

Technology gap

- You know what to do, but do you know which technique and technology to use?
- The techniques and technologies, that have ever been used to solve similar problems as yours, are more likely to work for your project
- Research and careful analysis are required to close the technology gaps in your project

Impact of gaps

- If you have any of these gaps (Knowledge, Skill and
- Technology) and do not know **how to deal with them**, you are not really ready to start the project
- Gap analysis and spiking
  - How **critical** are these gaps to project success?
  - Can you **close** these gaps? How to close them? A plan is required to spike each gap out …
  - Without an early spiking, the project is more likely to fail

Capturing gaps

- Along the development, more and more gaps will start to appear
- For example, the following new questions will come up:
  - Do we have a library that will do X for us?
  - Is there a framework that will solve Y for me?
  - Can we deliver the quality goals with our current technology?
- You need to ask yourself which gap is motivating you towards these questions
  - This will help you define the spiking plan

Weather Forecast Example

Problem, actor, pain point and goal

- **Case study problem:**
  - “Jerry lives in Melbourne and finds the weather patterns are odd. He is frustrated everyday since he either wears too many or too few clothes. He often gets wet as he is walking to work. This upsets him.”
- **Actor:** Jerry
- **Pain points of Jerry**
  - Frustration due to weather patterns
  - Sometimes Jerry gets wet because he didn’t dress for expected showers
- **GOAL:**
  - "Jerry is unable to dress adequately for the weather conditions in Melbourne. He needs the latest reliable weather forecast information before getting dressed & leaving home”

Problems to be solved

- Presenting the weather information in a way that Jerry can understand and act on it
- Select a reliable weather information provider, i.e. 95% accurate for next day forecast
- Define a mechanism to get the weather information from the provider (Cost?)
- Identify the frequency at which we retrieve the weather information. (Hourly? Daily? Weekly?) This frequency may change the reliability of information.

Some knowledge gaps

- Unclear about the following issues
  - What constitutes latest weather?
  - What is the metric for reliability/accuracy of weather forecast?
  - What will happen if we provide the wrong information? Will this be expensive?
  - How much information does Jerry need to dress properly?
  - Do we need to train Jerry in weather vocabulary and interpretation?
  - How do we deliver the information? (SMS/E-mail/Internet etc.)
- How critical are these gaps to project success?
- Plan to close the gaps …

Potential solutions

1. Change the weather patterns in Melbourne

2. Climate control the path Jerry takes to work

3. Brain wash Jerry to live with odd weather patterns

4. Provide Jerry with the latest weather forecast before he leaves home allowing him to dress appropriately

5. Teach Jerry to wear layers

Assumptions

- Some obvious **assumptions**
  - Jerry can read English and understand basic weather terminology
  - There is a source for weather forecast (i.e. we need not invent the technology and build the infrastructure)
- In order for the (chosen) solution to work, the following assumption is required even though we have no control of it
  - Jerry will make wise choices (regarding to what to wear) when provided with an accurate weather forecast
- Check the validity of the above assumptions

Solution attributes

- Ways of delivering weather information
  - SMS/E-mail
  - Internet
  - Secure
- Characteristics of the provided information
  - Accurate and reliable
  - Current
  - Simple and Clear
- All of these attributes reflect *decisions* made either by Jerry (i.e. user) or by the solution developer

Elaboration of the solution attributes

SMS/E-mail	The information is sent via SMS or E-mail to the user
Internet	User can register their interest via the internet, and also retrieve the latest weather information
Secure	The information stored about the user is encrypted, and access to it is restricted
Accurate/Reliable	The information provided is correct 95% of the time
Current	The information provided is for a 24 hours period (the current 24 hours period)
Simple and Clear	Common language should be used (I.e. no special meteorology words). When needed, the user can find out more information on the web site

More assumptions

- Jerry has a device capable of receiving an SMS or E-mail
- Jerry can register via the **Internet**
- A reliable weather forecast provider exists at a **reasonable price**
  - Bold words are still not clear and need to be refined, for example,
    - Internet → Web application
    - Reasonable price → less than 5c a day

Decisions made during solution refinement

- Weather forecast will be taken from the Bureau of Meteorology (BOM). Only freely available data from their web site shall be used http://www.bom.gov.au/vic/forecasts/melbourne.shtml
- Forecast information shall be sent via an SMS message at 6:00am in the morning. This will be based on the forecast issued at 5:00am by the BOM.

Some domain vocabulary

- BOM
- Weather forecast, Current forecast
- Weather terms: Rain, Showers, Fine, etc
- SMS
- Registered user

Data analysis

- What type of information (output) can allow the user to reach their goals?
  - Which scale of measurement does Jerry understand? (Celsius or Fahrenheit)
- What is the input that we need to produce this output, and can we get this input data?
  - Example: we need to provide the following information in the output:
    - Temperature range
    - Current temperature
    - Wind speed
    - Probability of rain/shower/precipitation, and when it is likely to occur
  - Sample output - “Rain in the morning. Fine after. 12°C– 24°C. Current temp: 12°C, Wind: 5 miles/hr north”
  - How do we fit the data from BOM into an SMS message? What data can we remove? What is the risk?

Data analysis (cont.)

- Can we provide this output within the overall scope and constraints identified so far?
  - What happens if Jerry moves to Sydney (or overseas)?
  - What happens if BOM stops providing free weather forecast services?
- What inputs are required from the users?
  - Information required to register Jerry
- Need to identify key inputs and outputs of the software
- Need to design test cases

Types of Requirements

- **Functional requirements**: Describe the interactions between the system and its environment independent from implementation
  - The watch system must display the time based on its location
- **Nonfunctional requirements**: User visible aspects of the system not directly related to functional behavior.
  - The response time must be less than 1 second
  - The accuracy must be within a second
  - The watch must be available 24 hours a day except from 2:00am - 2:01am and 3:00am - 3:01am
- **Constraints** ("Pseudo requirements"): Imposed by the client or the environment in which the system will operate
  - The implementation language must be COBOL

Properties of Requirements

- Correct
- Complete
- Consistent
- Clear, unambiguous
- Realistic
- Verifiable
- Traceable

Collecting data in the problem domain

- To understand the problem that we are dealing with, we can adopt the following techniques
  - Interview
  - Questionnaire
  - Experimentation by building a prototype
  - Observation
  - Document inspection
  - User story

User Story

- User Story
  - Users tell the stories, and developers listen, ask questions to understand context
  - A short description of the **behaviour** of the system
    - Generally implementation issues are not discussed
- The entire system is specified through stories
- Each story is short, goal oriented – **testable**
  - Can be used for usability testing
- Story is descriptive, but often a diagram or a sample output page helps to explain the concept much better
  - As in user interface snippets, forms or reports

Data to be collected and analyzed

- Actors - role and responsibility
- Scenarios/User Stories
- Source of the data that we have to deal with (reliability and accuracy, etc)
- Information that actors want (output)
- Software interfaces to existing actors
- What cause events to happen, and what cause work to be created
- Workflows and activities
- Knowing what type of problem it is (solvable?)

What is a User Story?

- A User Story is a requirement expressed from the perspective of an end-user goal.
- It's usually written out as a couple of sentences. Most user stories are written in the language of the users, so any user should be able to read a user story and immediately understand what it means.
- A User Story is really just a well-expressed requirement.
- User Story is only meant to describe a feature, but not describe *how* to implement it, meaning leaving out the technical aspect, it should describe the behavior or flow from user's perspective.

What is a User Story?

- The User Story format has become the most popular way of expressing requirements in Agile for a number of reasons:
  - It focuses on the viewpoint of a role who will use or be impacted by the solution
  - It defines the requirement in language that has meaning for that role
  - It helps to clarify the true reason for the requirement
- It helps to define high level requirements without necessarily going into low level detail too early

User Story

- User Stories are often deemed to comprise three elements - the 3C’s
- **C**ard
- **C**onversation
- **C**onfirmation

User Story

Card

- Stories are traditionally written on note cards.
- Cards may be annotated with estimates, notes, etc.

Conversation

- Details behind the story come out during conversations with product owner

Confirmation

- Acceptance tests confirm a story was coded correctly

User Story

As a user, I want to reserve a hotel room.

As a vacation traveler, I want to see photos of the hotels.

As a user, I want to cancel a reservation.

As a frequent flyer, I want to rebook a past trip so that I save time booking trips I take often.

Where are the details?

- As a user, I can cancel a reservation.
  - Does the user get a full or partial refund?
    - Is the refund to her credit card or is it site credit?
  - How far ahead must the reservation be cancelled?
    - Is that the same for all hotels?
    - For all site visitors? Can frequent travelers cancel later?
  - Is a confirmation provided to the user?
    - How?

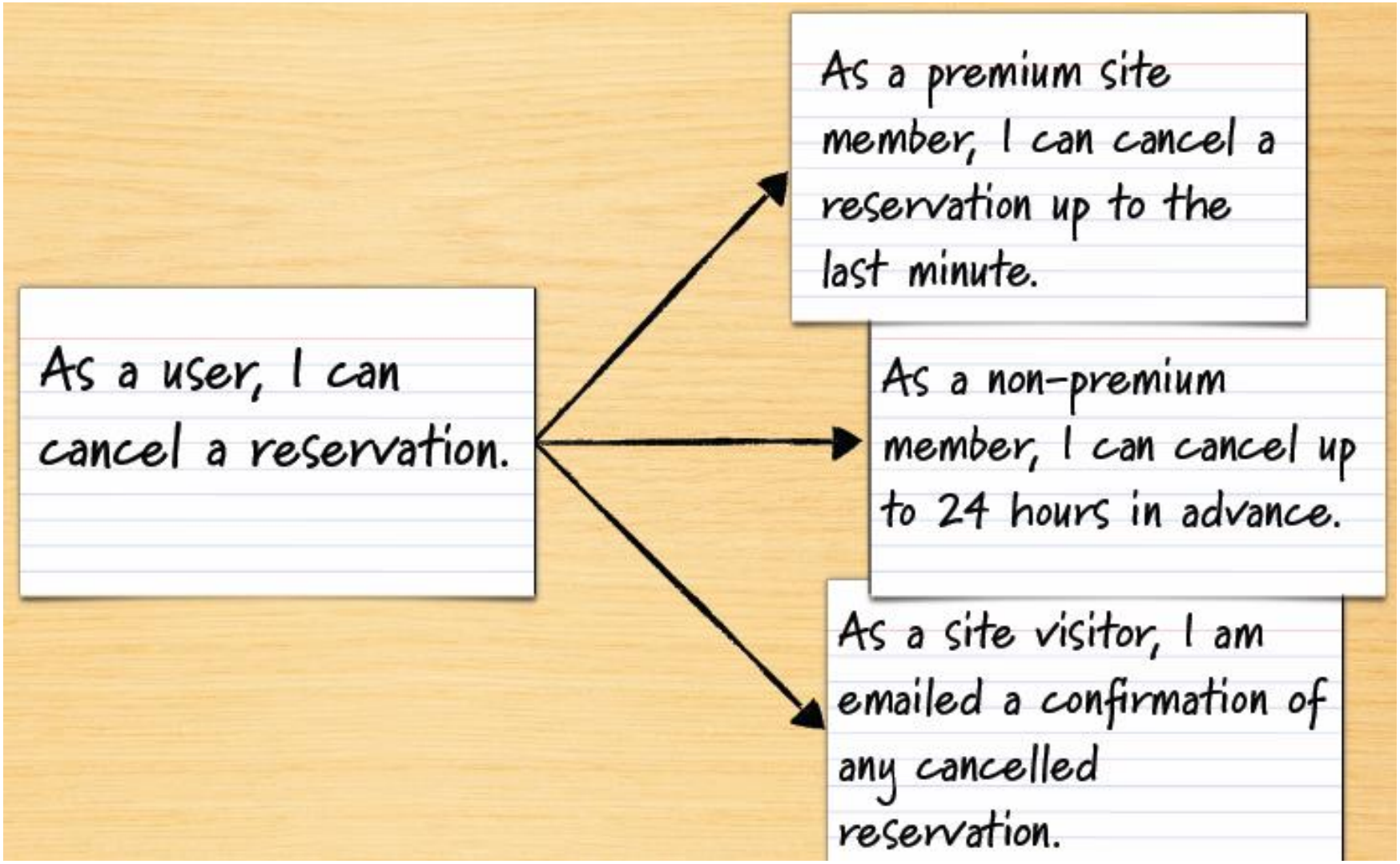
Details as conditions of satisfaction

As a user, I can cancel a reservation.

- The product owner's conditions of satisfaction can be added to a story
- These are essentially tests

- ☐ Verify that a premium member can cancel the same day without a fee.
- ☐ Verify that a non-premium member is charged 10% for a same-day cancellation.
- ☐ Verify that an email confirmation is sent.
- ☐ Verify that the hotel is notified of any cancellation.

Details added in smaller sub-stories



Logging in

- See how many user stories you can write about logging in.
- Examples:
  - As a registered user, I am required to log in so that I can access the system.
  - As a forgetful user, I can request a password reminder so that I can log in if I forget mine.

"As a <user role>,
I <want/need/can/
etc> <goal>
so that <reason>."

User Story format

- The format of the User Story is as follows:

As a < role>

I need <requirement or feature>

So that <goal / value>

User Story (Logging in)

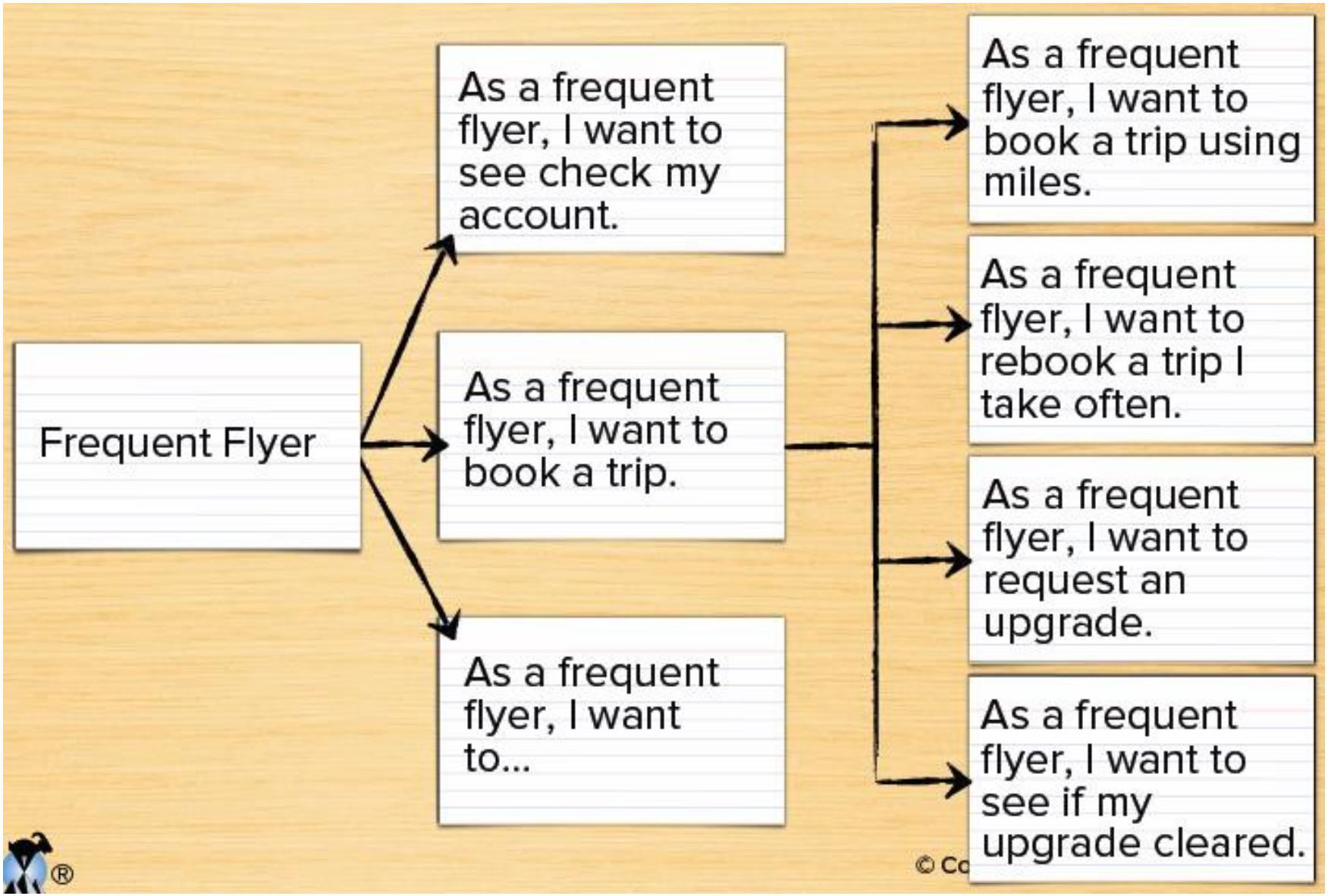
- **Confirmation**

1. Success – valid user logged in and refer to home page

- **a.** “Remember me” ticked – store cookie/automatic login next time.
- **b.** “Remember me” not ticked – force login next time.

2. Failure – display message:

- **a.** “Email address in wrong format”
- **b.** “Incorrect user name, please try again”
- **c.** “Incorrect password, please try again”
- **d.** “Service unavailable – please try again later”
- **e.** “Account has expired – refer to account renewal page”


Frequent Flyer
As a frequent flyer, I want to see check my account.
As a frequent flyer, I want to book a trip.
As a frequent flyer, I want to...
As a frequent flyer, I want to book a trip using miles.
As a frequent flyer, I want to rebook a trip I take often.
As a frequent flyer, I want to request an upgrade.
As a frequent flyer, I want to see if my upgrade cleared.

Words are imprecise

Entrée comes with soup or salad and bread.

Which is right?

- (Soup or Salad) and Bread
- (Soup) or (Salad and Bread)

Words are imprecise

As a user, I want to mow my lawn quickly and easily.

As a user, I want to be comfortable while mowing my lawn.

As

mc

an